

Memory, Storage & Numbers

Why this chapter matters: system design is a game of **numbers**. Is a design feasible? Do we need a cache, a CDN, 10 servers or 1,000? You can only answer with a feel for how fast memory, disks and networks are, how much data a system produces, and how to protect that data from failure. This chapter builds that intuition.

◀ QUICK RECALL — Chapter map

- 2.1 Memory hierarchy → 2.2 Locality & caches → 2.3 Virtual memory, page cache & durability
- 2.4 HDD / SSD / NVMe → 2.5 Block, file, object storage → 2.6 Sequential vs random I/O
- 2.7 RAID → 2.8 Beyond RAID: replication, erasure coding, backups
- 2.9 Units & powers of two → 2.10 Latency numbers → 2.11 IOPS vs throughput vs latency
- 2.12 Back-of-envelope method → 2.13 Worked examples → 2.14 Practice, Q&A, cheat sheet

◆ SENIOR INSIGHT — Builds on Chapter 1

- Chapter 1 showed that context switches and cache misses cost time. Here we see *why*: memory is ~200× slower than L1, and disks are millions of times slower.
- Thread pool tasks like `fs.readFile` exist precisely because disk I/O is so slow compared with CPU work.

2.1 The memory hierarchy — fast is small, big is slow

No technology is simultaneously fast, large and cheap. Computers solve this with a **memory hierarchy**: tiny, very fast memory close to the CPU, backed by larger, slower layers. Each layer acts as a **cache** for the layer below it.

fastest, smallest, most expensive per GB			
+-----+			
Registers	< 1 KB	~ 0.3 ns	
+-----+			
L1 cache	32-64 KB/core	~ 0.5-1 ns	
+-----+			
L2 cache	0.25-2 MB	~ 4-7 ns	
+-----+			
L3 cache	8-64+ MB	~ 10-20 ns	(shared by cores)
+-----+			
RAM	GBs - TBs	~ 100 ns	<-- volatile (lost on power off)
+-----+			
NVMe/SSD	100s GB - TBs	~ 10-150 us	<-- persistent
+-----+			
HDD	TBs	~ 5-10 ms	
+-----+			
Network	same DC ~0.5 ms	cross-continent ~150 ms	
+-----+			
slowest, largest, cheapest per GB			

Feel the gap — scale time so that 1 ns = 1 second

Operation	Real time	Human scale
L1 cache reference	0.5 ns	0.5 seconds
Branch mispredict	5 ns	5 seconds
L2 cache reference	7 ns	7 seconds
Mutex lock/unlock	25 ns	25 seconds
Main memory reference	100 ns	1.7 minutes
Compress 1 KB / send 1 KB over 1 Gbps	10 μ s	2.8 hours
SSD random read (4 KB)	150 μ s	1.7 days
Round trip inside one data center	500 μ s	5.8 days
Read 1 MB sequentially from SSD	1 ms	11.6 days
HDD seek	10 ms	3.8 months
Read 1 MB sequentially from HDD	30 ms	11 months
Packet California → Netherlands → California	150 ms	4.8 years

★ MUST REMEMBER

- Each step down the hierarchy is roughly **10x-1000x slower**. Memory $\approx$ 100 ns, SSD $\approx$ 100 μ s (1000x slower), HDD seek $\approx$ 10 ms (100x slower than SSD), cross-continent $\approx$ 150 ms.
- This one fact justifies caching, CDNs, read replicas, batching and data locality in every later chapter.

2.2 Locality of reference — why caches work

Caches work because programs are predictable. Two principles:

Principle	Meaning	Example
Temporal locality	Data used recently will likely be used again soon	Loop counter, hot user profile, popular product
Spatial locality	Data near recently used data will likely be used next	Iterating an array; the CPU loads a whole 64-byte cache line , not one byte

Consequences you can use in design:

- Arrays beat linked lists** for iteration (contiguous memory = fewer cache misses).
- Hit ratio** drives performance: average access time = hit% $\times$ cache time + miss% $\times$ slow time. With RAM 100 ns and SSD 100 μ s, a 99% hit ratio gives $\approx 0.99 \times 0.1 + 0.01 \times 100 \approx$ **1.1 μ s**; a 90% hit ratio gives $\approx$ **10 μ s** — 10x worse from just 9% more misses.
- 80/20 (Pareto) rule**: usually $\sim$ 20% of data serves $\sim$ 80% of traffic, so a small cache can absorb most reads (used again in Chapter 11).

◆ SENIOR INSIGHT — Hit ratio is non-linear

- Improving a cache from 90% to 99% hit ratio cuts misses 10x — often worth far more than making the cache itself faster. Always reason in **miss rate**, not hit rate.

2.3 Virtual memory, the page cache & durability

- Virtual memory**: each process sees its own private address space; the OS + MMU map it to physical RAM in **pages** (typically 4 KB). This is what gives processes isolation (Chapter 1).

- **Page fault:** accessing a page not in RAM. *Minor* = map it (cheap); *major* = read from disk (milliseconds).
- **Swap:** when RAM is full, the OS moves cold pages to disk. A server that swaps heavily becomes 1000x slower — servers are usually tuned to avoid swapping.
- **OOM killer** (Linux): when memory is exhausted, the kernel kills a process. In containers, a memory limit exceeded ⇒ the container is killed (exit 137).
- **Page cache:** the OS keeps recently read file data in free RAM. The second read of a file is served from memory. Databases also run their own **buffer pool**. This is why “free RAM” on a database server is never wasted.

◆ SENIOR INSIGHT — Durability: write to memory ≠ saved to disk

- A `write()` normally lands in the OS page cache and returns quickly. It survives a process crash but **can be lost on power failure** until flushed.
- `fsync()` forces data to the device — slower, but required for durability. Databases use **write-ahead logs (WAL)** + `fsync` at commit.
- Trade-off you can name in interviews: **fsync every write = safest, slowest; group commit / interval fsync = faster, small data-loss window** (e.g., Redis AOF everysec).

✖ COMMON MISTAKE — Node.js heap is not unlimited

- V8 caps the old-space heap (depends on version/system; roughly 2-4 GB by default on 64-bit). Raise it with `--max-old-space-size=4096`. Big in-process caches also add garbage-collection pauses — prefer Redis/Memcached for large caches (Chapter 11).

2.4 Storage devices: HDD, SSD, NVMe

	HDD (spinning disk)	SATA SSD	NVMe SSD
How it works	Spinning platters + moving read head (mechanical)	NAND flash chips, no moving parts, SATA interface	NAND flash over PCIe, many parallel queues
Random read latency	~5-10 ms (seek + rotation)	~50-150 μs	~10-100 μs
Random IOPS (approx.)	~100-200	~50k-100k	~500k-1M+
Sequential throughput	~100-250 MB/s	~500 MB/s	~3-7 GB/s
Cost per GB	Lowest	Medium	Highest of the three
Best for	Cold data, backups, archives, video, logs at large scale	General servers	Databases, caches, low-latency workloads

Figures are typical orders of magnitude, they vary by model and workload.

◆ SENIOR INSIGHT — SSD facts worth knowing

- Flash is written in pages but erased in larger **blocks**, so the SSD must relocate data — **write amplification**. Cells also wear out (limited program/erase cycles), so SSDs use **wear levelling** and over-provisioning; endurance is rated in TBW/DWPD.
- **Random vs sequential:** on HDD, random I/O is 100x+ slower. On SSD, the gap is small but sequential is still faster and gentler on wear.
- **TRIM** tells the SSD which blocks are free, keeping write performance stable.

🔑 **CORRECTION TO YOUR ORIGINAL NOTE** — Classic latency table is dated

- The popular “latency numbers” list your notes copy (HDD 30 MB/s, memory 4 GB/s) comes from ~2012. Modern HDDs do ~150+ MB/s and RAM does 10+ GB/s per channel.
- Do **not** memorise exact values — memorise the **ratios and orders of magnitude** (ns → μs → ms). Those are stable.

2.5 Block vs file vs object storage

Type	What you get	Access	Typical services	Best for
Block	Raw fixed-size blocks, like a virtual disk; you put a file system or database on it	Attached to ONE server (mostly); lowest latency	AWS EBS, GCP Persistent Disk, local NVMe	Databases, boot volumes
File	Hierarchical folders & files shared over a network	NFS / SMB; many servers share the same files	AWS EFS, Azure Files, NAS	Shared content, home dirs, legacy apps
Object	Flat namespace: object = data + metadata + unique key, accessed via HTTP API; immutable (replace whole object)	REST: PUT/GET/DELETE; virtually unlimited scale, higher latency (tens of ms)	AWS S3, GCS, Azure Blob, MinIO	Images, videos, backups, data lakes, static websites

Storage tiers (hot / warm / cold): keep frequently accessed data on fast, expensive media and move aging data to cheaper tiers (e.g., S3 Standard → Infrequent Access → Glacier archive) using lifecycle rules. Cost drops sharply, retrieval time rises (minutes-hours for archives).

★ **MUST REMEMBER**

- System design rule: **store files (images/videos) in object storage, store their metadata/URL in the database** — never large blobs in your relational DB.

2.6 Sequential vs random I/O — a core design idea

Moving the disk head (HDD) or managing flash blocks (SSD) makes **random access expensive and sequential access cheap**. On HDDs sequential can be 100× faster than random; even RAM prefers sequential (cache lines, prefetching).

Technique	How it exploits sequential I/O	Used by
Append-only log / WAL	Every change is appended to the end of a file first; the real data structure is updated later	PostgreSQL, MySQL (redo log), etcd
LSM trees	Buffer writes in memory, flush as sorted immutable files, compact in background — turns random writes into sequential writes	Cassandra, RocksDB, LevelDB, HBase
Batching & group commit	Combine many small writes into one large write / one fsync	Kafka, databases
Log-structured messaging	Messages stored in an append-only commit log; consumers read sequentially	Kafka

◆ **SENIOR INSIGHT** — Trade-off: B-tree vs LSM

- **B-tree** (most relational DBs): good reads and in-place updates, random writes. **LSM**: very high write throughput, reads may check several files (mitigated by bloom filters), background compaction costs I/O.
- Preview for Chapters 9-10: this is why write-heavy workloads often pick LSM-based stores.

2.7 RAID — Redundant Array of Independent Disks

RAID combines several physical disks into one logical unit to balance **performance, capacity and fault tolerance**. It protects against *disk failure* — nothing else.

Three building blocks: **Striping** (split data across disks → speed), **Mirroring** (copy data → redundancy), **Parity** (store a computed checksum → redundancy with less space).

Level	Strategy	Min disks	Usable capacity (n disks)	Survives	Performance	Typical use
RAID 0	Striping	2	n (100%)	0 failures — one disk dies, all data lost	Best read & write	Scratch space, caches, temp data
RAID 1	Mirroring	2	n / 2 (50%)	1 disk per mirror	Good reads, writes at single-disk speed	Boot disks, small critical data
RAID 5	Striping + 1 parity	3	n – 1	Any 1 disk	Good reads; small writes slow (~4 I/Os)	Read-heavy file servers
RAID 6	Striping + 2 parity	4	n – 2	Any 2 disks	Good reads; writes slower (~6 I/Os)	Large arrays, archives
RAID 10	Mirror + stripe (1+0)	4	n / 2 (50%)	1 per mirror pair (up to n/2 if in different pairs)	Excellent reads & writes	Databases, high-IOPS workloads

Example: 4 disks × 2 TB

RAID 0	RAID 1 / 10	RAID 5	RAID 6
8 TB usable, no protection	4 TB usable (two mirrored pairs striped)	6 TB usable, survives 1 failure	4 TB usable, survives any 2 failures

How parity works (XOR)

Parity = A XOR B. If a disk is lost, XOR the survivors to rebuild it.

```
Disk A = 1010
Disk B = 0110
Parity = A XOR B = 1100          (stored on a third disk)

Disk A fails -> A = B XOR Parity = 0110 XOR 1100 = 1010  (recovered!)
```

◆ SENIOR INSIGHT — Why RAID 5 is risky on big HDDs, and the write penalty

- **Write penalty:** updating one block in RAID 5 = read old data + read old parity + write new data + write new parity (4 I/Os). RAID 6 ≈ 6 I/Os. RAID 10 = 2 writes.
- **Rebuild risk:** rebuilding a failed multi-TB disk takes hours/days, stressing all remaining disks. A second failure or an unrecoverable read error during rebuild destroys a RAID 5 array. This is why RAID 6 or RAID 10 is preferred for large disks and databases.
- **Hardware vs software RAID:** hardware controllers have cache + battery backup; software RAID (Linux mdadm, ZFS) is flexible and common today. Cloud block storage (EBS) is already replicated for you, but that does not replace backups.

★ MUST REMEMBER — RAID is not a backup

- RAID does not protect against **accidental deletion, corruption, ransomware, bugs, controller failure, or a fire/flood** — all of these happen to every disk at once.
- Backup rule **3-2-1**: 3 copies of data, on 2 different media, 1 off-site (modern addition: 1 immutable/offline copy).

2.8 Beyond RAID: replication, erasure coding & backups

RAID protects one machine. Large systems protect data **across machines, racks and regions**.

Technique	How	Space overhead	Notes
Replication	Store N full copies on different nodes/racks/zones (typically N = 3)	3× (200% extra)	Simple, fast reads & repairs; used by HDFS, Cassandra, Kafka, DB replicas
Erasure coding	Split data into k data + m parity chunks on different nodes; any k chunks rebuild the data (Reed-Solomon)	$(k+m)/k$, e.g., $10+4 \rightarrow 1.4\times$	Survives m losses ($10+4$ survives 4) at less than half the cost of 3× replication; CPU + network heavier to read/repair. Used by object stores (S3-like), HDFS EC
Backup / snapshot	Point-in-time copy stored elsewhere	Depends (incremental)	Protects against logical errors & disasters; replication would faithfully copy a deletion!

◆ SENIOR INSIGHT — RPO and RTO — the two numbers behind every DR plan

- **RPO** (Recovery Point Objective): how much data you can afford to lose, measured in time (RPO = 5 min $\Rightarrow$ backups/replication at least every 5 min).
- **RTO** (Recovery Time Objective): how long you can be down before service is restored.
- Lower RPO/RTO $\Rightarrow$ more cost: synchronous multi-region replication + hot standby (RPO $\approx$ 0, RTO minutes) vs nightly backup (RPO 24 h, RTO hours). Revisited in Chapter 5.

2.9 Units, powers of two & handy conversions

Power	Exact value	Approx.	Size	Remember as
2^7	128			Signed byte max 127
2^8	256			Values in 1 byte
2^{10}	1,024	1 thousand	1 KB	Kilo
2^{16}	65,536		64 KB	Number of TCP/UDP ports
2^{20}	1,048,576	1 million	1 MB	Mega
2^{30}	1,073,741,824	1 billion	1 GB	Giga
2^{32}	4,294,967,296	4.3 billion	4 GB	IPv4 addresses; 32-bit int range
2^{40}	1,099,511,627,776	1 trillion	1 TB	Tera
2^{50}	$\sim 1.13 \times 10^{15}$	1 quadrillion	1 PB	Peta
2^{64}	$\sim 1.8 \times 10^{19}$		16 EB	64-bit address space

Time conversions (memorise)

Period	Seconds	Rounded for estimates
1 day	86,400	$\approx 10^5$ (use 100,000)
1 month	~ 2.6 million	$\approx 2.5 \times 10^6$
1 year	~ 31.5 million	$\approx 3 \times 10^7$

Requests/day → requests/second (QPS): divide by $\sim 10^5$

Per day	1 M	10 M	100 M	1 B
≈ Per second	~12	~115	~1,150	~11,600

Typical data sizes (rough)

Item	Size	Item	Size
ASCII char	1 byte (UTF-8: 1-4)	Tweet / chat message (text + metadata)	~300 B - 1 KB
int / float	4 bytes	DB row (user, order)	~0.5 - 2 KB
long / timestamp / double	8 bytes	Photo (compressed)	0.2 - 3 MB
UUID	16 bytes (36 as text)	1 min HD video (~5 Mbps)	~35-40 MB
IPv4 / IPv6	4 B / 16 B	Average web page	~2 MB

✖ COMMON MISTAKE — Bits vs bytes, KB vs KiB

- Network speed is in **bits/s**, storage in **bytes**. **1 Gbps = 125 MB/s** theoretical (≈ 100 MB/s real). Divide Mbps by 8 to get MB/s.
- KB = 1000 B (disk vendors), KiB = 1024 B (OS). For estimation, treat 1 K $\approx 1,000$ — precision does not matter, **order of magnitude** does.

2.10 Latency numbers every engineer should know

Operation	ns	μs	ms	Insight
L1 cache reference	0.5			
Branch mispredict	5			
L2 cache reference	7			14× L1
Mutex lock/unlock	25			Uncontended
Main memory reference	100	0.1		20× L2, 200× L1
Compress 1 KB (Zippy/Snappy)	10,000	10		
Send 1 KB over 1 Gbps network	10,000	10		
Read 4 KB randomly from SSD	150,000	150	0.15	~1 GB/s SSD (classic)
Read 1 MB sequentially from memory	250,000	250	0.25	Classic ~4 GB/s
Round trip within same data center	500,000	500	0.5	
Read 1 MB sequentially from SSD	1,000,000	1,000	1	4× memory
HDD seek	10,000,000	10,000	10	20× data-center round trip
Read 1 MB sequentially from 1 Gbps network	10,000,000	10,000	10	40× memory, 10× SSD
Read 1 MB sequentially from HDD	30,000,000	30,000	30	120× memory, 30× SSD
Packet CA → Netherlands → CA	150,000,000	150,000	150	Speed-of-light limit

Unit reminders: 1 ns = 10^{-9} s; 1 μs = 10^{-6} s = 1,000 ns; 1 ms = 10^{-3} s = 1,000 μs = 1,000,000 ns.

Handy metrics (classic): sequential read — HDD ≈ 30 MB/s (modern 150+), 1 Gbps Ethernet ≈ 100 MB/s, SSD ≈ 1 GB/s, memory ≈ 4 GB/s (modern 10+). World-wide round trips: only **6-7 per second**; inside one data center: **~2,000 per second**.

★ **MUST REMEMBER** — Design rules that fall out of these numbers

- **Memory is fast, disk is slow** → cache hot data in RAM. **Avoid disk seeks** → prefer sequential access, SSDs, indexes.
- **Compress before sending** over the network when CPU is cheaper than bandwidth (compression $\approx 10 \mu\text{s}/\text{KB}$ vs sending $1 \text{ KB} \approx 10 \mu\text{s}$ on 1 Gbps ; wins on slower links).
- **Minimise round trips** → batch requests, use connection pooling, keep chatty services in the same region.
- **Move data closer to users** → CDN, geo-distributed replicas (a cross-continent hop costs 150 ms no matter how big the server is).
- **Speed of light is a hard limit**: $\sim 5 \mu\text{s}$ per km in fibre, so Delhi ↔ California can never beat roughly $100+$ ms round trip.

2.11 Latency vs throughput vs IOPS vs bandwidth

Metric	Question it answers	Unit	Analogy (highway)
Latency	How long does ONE operation take?	ns / μs / ms	Time for one car to travel A→B
Bandwidth	Maximum capacity of the link	bits/s or bytes/s	Number of lanes
Throughput	Work actually completed per unit time	req/s, MB/s	Cars arriving per hour
IOPS	I/O operations per second a device sustains	ops/s	Cars entering per second

★ **MUST REMEMBER**

- **Throughput (MB/s) = IOPS × I/O size**. Example: $10,000 \text{ IOPS} \times 4 \text{ KB} = 40 \text{ MB/s}$; but $500 \text{ IOPS} \times 1 \text{ MB} = 500 \text{ MB/s}$ — small random I/O is IOPS-limited, large sequential I/O is bandwidth-limited.
- **Little's Law**: concurrent requests in the system $L = \text{arrival rate } \lambda \times \text{average latency } W$. Example: $2,000 \text{ req/s} \times 50 \text{ ms} = 100 \text{ requests in flight}$ — size thread pools and connection pools from this.

◆ **SENIOR INSIGHT** — Tail latency (p99) matters more than average

- If one user request fans out to 100 backend calls, even 1% slow calls mean $\sim 63\%$ of requests hit at least one slow call ($1 - 0.99^{100}$). Track p50, p95, p99, p99.9 — not just the mean.
- A system can have high throughput and bad latency, or the reverse (e.g., batching raises throughput but also latency). Always state which you optimise.

2.12 Back-of-the-envelope estimation — the method

Interviewers use estimation to check that you can size a system and spot bottlenecks. The goal is the **right order of magnitude** in 5 minutes, not exact figures.

- 1 **State assumptions** out loud (users, DAU, actions per user, object size, read:write ratio, retention). Interviewers grade the reasoning.
- 2 **Round aggressively** (use 10^5 s/day , $1\text{K} \approx 1,000$). Work in powers of 10.
- 3 **Compute QPS** → peak QPS → storage → bandwidth → memory/cache → servers.
- 4 **Sanity-check** the result and conclude what it means for the design (need sharding? CDN? cache?).

Quantity	Formula
Average QPS	$\text{DAU} \times \text{actions per user per day} \div 86,400 (\approx \div 10^5)$
Peak QPS	Average QPS × peak factor (commonly 2-3×; spiky apps like events/sales much more)
Read QPS vs Write QPS	Total QPS split by read:write ratio (e.g., 10:1 for read-heavy, social feeds 100:1+)
Storage	$\text{New records/day} \times \text{bytes/record} \times 365 \times \text{years of retention} \times \text{replication factor}$

Quantity	Formula
Bandwidth	$\text{QPS} \times \text{payload size (ingress = writes, egress = reads)}; \times 8 \text{ for bits/s}$
Cache memory	Daily read data $\times 20\%$ (Pareto rule) — then check it fits in a few servers' RAM
Servers	$\text{Peak QPS} \div \text{per-server capacity}$ (then add 30–50% headroom and redundancy: $N+1 / N+2$)

◆ **SENIOR INSIGHT** — **Rough capacity of one machine (order of magnitude only, workload dependent!)**

- Web/app server (Node, JVM) doing real work: hundreds to a few thousand req/s. Nginx serving static/proxy: tens of thousands req/s.
- Redis: $\sim 100\text{k ops/s}$ per instance (single-threaded core). Single relational DB: a few thousand to $\sim 10\text{k+}$ simple queries/s; writes much lower.
- Always **benchmark** your real workload. Use these only to decide “1 server, 10 servers or 1,000?”

2.13 Worked examples

Example A — URL shortener (assumptions: 100 M new URLs per month, read:write = 10:1, 500 bytes per record, keep 5 years)

Step	Calculation	Result
Write QPS	$100 \text{ M} \div (30 \times 86,400 \approx 2.6 \text{ M s})$	$\approx 40 \text{ writes/s}$ (peak $\times 2 \approx 80$)
Read QPS	40×10	$\approx 400 \text{ reads/s}$ (peak ≈ 800)
Total records	$100 \text{ M} \times 12 \times 5$	6 billion
Storage (5 yrs)	$6 \text{ B} \times 500 \text{ B}$	$\approx 3 \text{ TB}$ ($\times 3$ replicas $\approx 9 \text{ TB}$)
Read bandwidth	$400/\text{s} \times 500 \text{ B}$	$\approx 200 \text{ KB/s} \approx 1.6 \text{ Mbps}$ (tiny)
Cache size	$400/\text{s} \times 86,400 = 34.6 \text{ M reads/day} \times 500 \text{ B} \approx 17 \text{ GB}$; keep 20%	$\approx 3.5 \text{ GB}$ fits in one Redis node
Conclusion		Low traffic; one primary DB + replicas + a cache is enough. The hard part is generating unique short IDs, not capacity.

Example B — Photo sharing app (10 M DAU, 20% upload one 2 MB photo daily, each user views 50 photos/day, thumbnails/feed images $\sim 200 \text{ KB}$)

Step	Calculation	Result
Uploads/day	$10 \text{ M} \times 0.2$	2 M photos/day $\approx 23 \text{ uploads/s}$
Storage/day	$2 \text{ M} \times 2 \text{ MB}$	4 TB/day
Storage/year	$4 \text{ TB} \times 365$	$\approx 1.5 \text{ PB/year}$; $\times 3$ replication $\approx 4.4 \text{ PB}$ (erasure coding $1.4\times \approx 2 \text{ PB}$)
Views/day	$10 \text{ M} \times 50$	500 M $\approx 5,800 \text{ reads/s}$ (peak $\approx 12\text{k} - 17\text{k}$)
Egress bandwidth	$5,800/\text{s} \times 200 \text{ KB}$	$\approx 1.16 \text{ GB/s} \approx 9.3 \text{ Gbps}$ average
Conclusion		Needs object storage (S3-style) for photos, a CDN to serve $\sim 10 \text{ Gbps}$, and image resizing pipelines (async queues). A single server cannot serve this.

✖ COMMON MISTAKE — Estimation traps

- Forgetting **replication** (×3) and **indexes/metadata overhead** (+20-50%) in storage.
- Using **average** QPS to size servers — size for **peak** and keep headroom.
- Mixing bits and bytes in bandwidth; mixing per-day with per-second.
- Giving a number without stating the assumptions behind it.

2.14 Practice questions (try before looking)

P1. A service has 50 M DAU, each makes 20 requests/day. Average and peak QPS (peak ×3)?

Answer: $50 \text{ M} \times 20 = 1 \text{ B/day} \div 10^5 \approx 10,000 \text{ QPS average (11.6k exact); peak} \approx \mathbf{30,000 \text{ QPS}}$.

P2. Chat app: 100 M messages/day, 200 B each, keep 1 year, replication 3. Storage?

Answer: $100 \text{ M} \times 200 \text{ B} = 20 \text{ GB/day} \times 365 = 7.3 \text{ TB} \times 3 \approx \mathbf{22 \text{ TB}}$ (plus index overhead).

P3. Video: 1 M uploads/day, each 100 MB. Storage per day and per year?

Answer: $100 \text{ TB/day; } \approx \mathbf{36.5 \text{ PB/year}}$ before transcoding copies and replication.

P4. Your DB disk does 20,000 IOPS with 8 KB pages. Max random-read throughput?

Answer: $20,000 \times 8 \text{ KB} = \mathbf{160 \text{ MB/s}}$.

P5. RAID 5 over 6 × 4 TB disks. Usable capacity and failures survived?

Answer: $(6-1) \times 4 = \mathbf{20 \text{ TB}}$, survives 1 disk. (RAID 6 → 16 TB, survives 2.)

Interview Q&A (30-second answers)

Q1. Why is an SSD so much faster than an HDD for random reads?

No mechanical seek/rotation: flash is addressed electronically, so latency is $\sim 100 \mu\text{s}$ instead of $\sim 10 \text{ ms}$ and it handles many parallel requests.

Q2. Why are sequential writes faster, and how do databases use that?

Avoids seeks and lets the device prefetch/merge. Databases append to a WAL / use LSM trees so writes are sequential, updating data structures later.

Q3. RAID 5 or RAID 10 for a write-heavy database?

RAID 10: no parity write penalty, faster rebuilds, better random-write performance; costs 50% capacity.

Q4. Is RAID a backup?

No. It only handles disk failure. Deletions, corruption, ransomware and disasters are replicated instantly. Need versioned, off-site backups (3-2-1).

Q5. Why not just store everything in RAM?

Cost per GB, capacity limits and volatility. Use RAM for hot data (cache) and durable storage for the source of truth.

Q6. What does fsync do and why does it matter?

Forces buffered writes to stable storage. Without it, acknowledged writes can be lost on power failure; with it every write is slower — hence group commit.

Q7. Replication vs erasure coding?

Replication = N full copies: simple, fast, $3\times$ space. Erasure coding = $k+m$ chunks: $\sim 1.4\times$ space with similar durability, but more CPU/network on reads and repairs.

Q8. How do you estimate servers needed?

$\text{DAU} \times \text{actions} \div 10^5 = \text{QPS}$, $\times$ peak factor, $\div$ per-server capacity (benchmarked), + headroom and redundancy.

Chapter 2 Cheat Sheet

◀ QUICK RECALL — Revise this in 2 minutes

- **Hierarchy:** registers → L1 → L2 → L3 → RAM → SSD → HDD → network. Each step $\approx 10\text{--}1000\times$ slower.
- **Key numbers:** RAM 100 ns • SSD 150 μs • DC round trip 0.5 ms • HDD seek 10 ms • cross-continent 150 ms.
- **Caches work** via temporal + spatial locality; improve **miss rate**, not just hit rate.
- **Write $\neq$ durable** until fsync; WAL + group commit balance safety and speed.
- **Sequential $\gg$ random** I/O → WAL, LSM, Kafka logs, batching.
- **Storage:** block (DB disks), file (shared NFS), object (S3: images/videos/backups). Metadata in DB, blobs in object store.
- **RAID:** 0 = speed only • 1 = mirror • 5 = 1 parity ($n-1$) • 6 = 2 parity ($n-2$) • 10 = mirror+stripe (best for DBs). **RAID $\neq$ backup.**
- **Durability at scale:** $3\times$ replication or erasure coding (e.g., $10+4 = 1.4\times$) + backups (3-2-1) + RPO/RTO.
- **Units:** $2^{10}=\text{KB}$, $2^{20}=\text{MB}$, $2^{30}=\text{GB}$, $2^{40}=\text{TB}$; 1 day $\approx 10^5$ s; 1 Gbps = 125 MB/s.
- **Estimation:** QPS = $\text{DAU} \times \text{actions} \div 10^5$; peak $\times 2\text{--}3$; storage = size $\times$ count $\times$ retention $\times$ replicas; cache = 20% of daily reads.
- **Throughput = IOPS $\times$ size. Little's Law: $L = \lambda \times W$. Watch p99, not the mean.**